

NORTH SOUTH UNIVERSITY
Department of Electrical and Computer Engineering

Senior Design Project (CSE499B)

Summer 2026

Sustainability and Environmental Effects

AI-Powered Voice-Based Medical Pre-Screening System with Intelligent Symptom Capture and Clinical Documentation for Bangladesh

(AI Medical Pre-Screening Assistant)

Submitted by

Rafiur Rahman Mashrafi (ID: 2221971042)

M. G. Rabbi Hossen (ID: 2222516042)

Israt Zaman Srity (ID: 2211084042)

Faculty Advisor

Dr. Mohammad Ashrafuzzaman Khan [AzK]
Department of Electrical and Computer Engineering

1 Sustainability and Environmental Effects

1.1 Introduction

Every software system carries an environmental cost that is easy to overlook because it is not visible at the point of use. Running code consumes electricity through the processors, memory, storage, and network hardware it drives; that electricity is produced by an energy mix that, in most of the world, still emits greenhouse gases. For an artificial-intelligence (AI) system this chain is amplified, because model training and continuous inference are computationally heavy compared with conventional applications [1, 2]. This chapter analyses the sustainability implications of *this specific project*—an AI-powered, voice-based medical pre-screening system for Bangladeshi clinics—rather than sustainability in the abstract. It connects recognised green-software principles to the team’s actual technical choices: a FastAPI backend, a fifteen-module speech-and-language pipeline in which several stages run as local deterministic code, a multi-provider free-tier strategy for the remaining language-model calls, an embedded database, and lightweight browser-based kiosk and dashboard interfaces running on modest clinic hardware. Because the system is intended for real deployment in small, low-resource clinics, energy efficiency is not merely an ethical add-on; it is a functional requirement that overlaps directly with the project’s cost and hardware constraints. No measured energy figures for the team’s own system exist yet; all quantitative values in this chapter are therefore clearly framed as external benchmarks or estimates, not measured project data.

1.2 How AI and Software Systems Consume Energy

The environmental footprint of an AI product accumulates across three distinct phases, and it is useful to separate them because the team can influence each one differently.

- **Production / development phase.** This covers writing code and, crucially for AI, *training or fine-tuning models*. Training is the most energy-dense activity in a machine-learning lifecycle: it repeatedly processes large datasets on power-hungry accelerators over hours or days. A widely cited study estimated that training a single large natural-language-processing (NLP) model can emit roughly as much carbon dioxide as the lifetime emissions of five average cars [2], and the compute used for state-of-the-art results grew by about $300,000\times$ over six years [1].
- **Runtime / inference phase.** Once deployed, the system consumes energy every time it processes a patient. Individually each inference is small, but at scale inference dominates the lifecycle energy of a deployed model, and industry analyses attribute the majority of AI energy demand to inference rather than training [6, 4]. For a clinic tool that runs speech recognition, text normalisation, entity extraction, and risk assessment on *every* patient visit, the per-consultation cost is the figure that matters most.
- **Hidden infrastructure.** Beyond the visible application sit data centres, cloud servers, storage, and networks, plus the “embodied” emissions of manufacturing the hardware itself. Data centres already account for roughly 1.5% of global electricity use—about 415 TWh in 2024—and this is projected to more than double toward 945 TWh by 2030, driven largely by AI workloads [6]. An always-on backend and any hosted service draw power continuously, even when no consultation is in progress.

1.3 Environmental Footprint of This Project

Table 1 maps the project’s confirmed technology stack to its dominant energy characteristic and to the phase in which that energy is spent. The system is a fifteen-module pipeline (with a deliberate numbering gap at Module 5, whose emergency-detection role was consolidated into the Module 10 risk-assessment engine under design decision ADR-0024).

Several observations follow directly from the team’s design. First, GPU work is confined to short, parameter-efficient fine-tuning on a *shared* cloud T4 rather than full-model training from scratch, which keeps development-phase energy modest. Second, and most importantly for a per-consultation footprint, a substantial part of the pipeline runs as *local deterministic code with no language-model call at all*:

Table 1: Energy profile of the project’s technology stack.

Component	Energy characteristic	Dominant phase
Planned LoRA fine-tuning of a compact ASR model on a shared cloud GPU (Colab T4)	Bursty, high-intensity GPU compute; one-off but repeated across experiments and retraining rounds	Development
Real-time ASR (browser Web Speech API now; <i>faster-whisper</i> int8 on CPU planned)	Recurring per-consultation compute on the patient’s own device	Runtime
Language-model calls (extraction, summary, follow-up, risk) via shared free-tier providers	Recurring per-consultation inference, offloaded to third-party data centres	Runtime + hidden
Local deterministic modules (completion check, red-flag rule, XAI fallback, report assembly)	Negligible; ordinary CPU logic with no model inference at all	Runtime
Embedded SQLite EHR database (config-driven path to PostgreSQL at scale)	Very low idle overhead; file-based, no separate server process	Hidden infrastructure
FastAPI backend + static HTML/JavaScript kiosk and dashboards	Always-on server power; lightweight front-end with no heavy client framework	Hidden infrastructure

the completion check (Module 9), the safety-critical red-flag rule inside Module 10, the explainable-AI fallback (Module 11), and the clinical-report assembly (Module 12) are all computed locally. This means the safety and reporting core keeps working even during a total outage of every language-model provider, and it removes those stages from the energy-hungry inference budget entirely. Third, the language-model calls that *do* remain are spread across shared free-tier providers under design decision ADR-0026—a high-frequency extraction tier, a quality tier for summaries and risk, and a low-latency tier for the live follow-up loop, with a fallback provider—so the project reuses existing, highly utilised data-centre capacity instead of self-hosting additional GPUs. Finally, the always-on components are deliberately lean: an embedded, file-based SQLite database adds negligible idle overhead compared with a continuously running database server, and the kiosk and dashboards are plain static HTML and JavaScript rather than a heavyweight single-page framework.

1.4 Measuring and Estimating Impact

A recognised difficulty in green software engineering is that software’s carbon footprint cannot be read off a meter; it must be estimated from the hardware it drives, the electricity that hardware consumes, and the carbon intensity of the local grid [7]. For this project no direct energy measurements exist yet, so the team’s approach is to *estimate* rather than assert. Two estimation strategies are relevant to the stack:

- **Training energy.** GPU training energy can be approximated from the accelerator’s rated power, the training duration, and a data-centre power-usage-effectiveness (PUE) multiplier, then converted to emissions using a grid carbon-intensity factor. Tools such as the ML CO₂ Impact calculator formalise exactly this computation for cloud GPU jobs [3] and are well suited to logging the cost of each planned LoRA fine-tuning run on Colab.
- **Per-consultation inference.** The team already records per-module latency: the first end-to-end live pipeline run logged stage latencies of roughly 0.5–8.6 s in a `module_events` table. Latency is not energy, but this timing instrumentation is precisely the hook needed to estimate energy per consultation—measuring device power over one representative visit and multiplying by expected

daily volume. The Green Software Foundation’s Software Carbon Intensity (SCI) standard—now ISO/IEC 21031:2024—is the appropriate framing: it expresses impact as a *rate* per functional unit (here, “per completed pre-screening”), computed as operational emissions (energy $\times$ grid carbon intensity) plus a share of hardware embodied emissions [5].

Grounding these estimates in Bangladesh matters: the national grid is comparatively carbon-intensive because it is dominated by natural gas and other fossil sources, so the *same* computation emits more here than it would on a low-carbon grid. This strengthens rather than weakens the case for local, on-device efficiency.

1.5 Sustainable Design Choices: The Seven Rs Applied

The reference framework adapted here is the “seven Rs” of sustainable software (Refuse, Reduce, Reuse, Repurpose, Repair, Rot, Recycle) [7], which complements the SCI standard’s measurement view with a set of design actions. Table 2 maps each R onto concrete decisions already embedded in the project, avoiding categories that do not apply.

Table 2: The seven Rs mapped to concrete project decisions.

Principle	Applied action in this project
Refuse	Avoid oversized general-purpose LLMs where a small task-specific model suffices; the ASR backbone is a compact model, and safety, completion, and reporting logic use no model at all.
Reduce	Run Modules 9, 10 (red-flag rule), 11 (fallback), and 12 as local deterministic code—no inference; use LoRA parameter-efficient fine-tuning; and, per ADR-0024, fold the retired emergency module into Module 10 to remove one model call per consultation.
Reuse	Build on pre-trained open models (<i>faster-whisper</i> , Bangla ASR/NLP models) rather than training from scratch; reuse shared free-tier inference (ADR-0026: Gemini Flash / Flash-Lite, Groq, OpenRouter fallback) instead of self-hosting extra GPUs.
Repurpose	The Bangla speech and NLP resources produced along the way are reusable assets for future healthcare-AI work beyond this system.
Repair	Doctor feedback (Module 15) drives periodic, batched offline retraining with a regression check before any update ships, rather than wasteful continuous retraining.
Rot	Apply EHR data-retention/archival policies and database indexing so storage does not grow without bound; the embedded SQLite store keeps idle overhead minimal.
Recycle	Target commodity, low-resource, CPU-only hardware for clinic kiosks, extending the usable life of existing devices and reducing embodied-emission churn.

The single most impactful lever is model right-sizing. Keeping the safety and reporting core as local logic, and deploying quantised models (for example *faster-whisper* in int8) on CPU-only clinic hardware rather than routing every consultation to a large cloud model, keeps per-inference energy low and, conveniently, aligns with the project’s stated requirement to run on low-resource devices. Recent work confirms that task-appropriate small models—quantised and run locally—can cut inference energy by roughly an order of magnitude versus large hosted alternatives, while also keeping sensitive patient data on-device [4].

1.6 Trade-offs and Future Considerations

Sustainability decisions in this project are genuine trade-offs, not free wins.

- **Accuracy versus energy.** Larger models generally transcribe dialectal Bangla and Banglish more accurately, but cost more energy per consultation. The team’s position is to fine-tune a compact model for the target domain rather than scale up indiscriminately, treating efficiency as a first-class evaluation criterion alongside word-error rate, in the spirit of Green AI [1].
- **Cloud versus on-device.** Cloud inference offers accuracy and elastic capacity but depends on reliable connectivity and concentrates energy in data centres [6]; on-device inference is resilient to the

intermittent power and connectivity typical of rural Bangladeshi clinics and avoids repeated network transfers, at the cost of local compute limits. The swappable-provider design and the local-first safety core let a clinic choose the appropriate point on this spectrum.

- **Convenience versus retention.** Storing every transcript indefinitely aids auditability and longitudinal care but steadily grows the stored-data footprint; disciplined retention and archival policies are the counterweight.

Looking ahead, the team plans to log the energy and estimated emissions of each fine-tuning run, extend the existing per-module latency instrumentation into a per-consultation SCI-style metric once the pipeline is stable, and prefer carbon-aware scheduling for any batched retraining (running heavy jobs when the grid is cleaner or off-peak).

1.7 Conclusion

Sustainability for this project is inseparable from its core goal: a tool that a small, low-resource Bangladeshi clinic can actually install and run. The same choices that make the system deployable on modest, CPU-only hardware—parameter-efficient fine-tuning, keeping safety and reporting logic as local deterministic code, quantised local inference, reuse of open pre-trained models and shared free-tier providers, consolidation of redundant modules, a lightweight embedded database, and disciplined data retention—are precisely the choices that reduce its energy and carbon footprint. Because no measured figures yet exist, the responsible next step is measurement: estimating training energy with established calculators and tracking a per-consultation carbon-intensity metric against the ISO/IEC 21031 standard. Framed this way, environmental responsibility and the project’s real-world deployment objective point in the same direction, and the team is committed to keeping efficiency a first-class design criterion through to deployment.

References

- [1] R. Schwartz, J. Dodge, N. A. Smith, and O. Etzioni, “Green AI,” *Communications of the ACM*, vol. 63, no. 12, pp. 54–63, 2020.
- [2] E. Strubell, A. Ganesh, and A. McCallum, “Energy and Policy Considerations for Deep Learning in NLP,” in *Proc. 57th Annual Meeting of the Association for Computational Linguistics (ACL)*, Florence, Italy, 2019, pp. 3645–3650.
- [3] A. Lacoste, A. Luccioni, V. Schmidt, and T. Dandres, “Quantifying the Carbon Emissions of Machine Learning,” *arXiv preprint arXiv:1910.09700*, 2019.
- [4] A. S. Luccioni, Y. Jernite, and E. Strubell, “Power Hungry Processing: Watts Driving the Cost of AI Deployment?,” in *Proc. ACM Conf. on Fairness, Accountability, and Transparency (FAccT)*, 2024. (arXiv:2311.16863).
- [5] Green Software Foundation, “Software Carbon Intensity (SCI) Specification,” ISO/IEC 21031:2024, 2024. [Online]. Available: <https://greensoftware.foundation/standards/sci/>
- [6] International Energy Agency, “Energy and AI,” IEA Special Report, 2025. [Online]. Available: <https://www.iea.org/reports/energy-and-ai>
- [7] M. Accorinti, “Environmental Sustainability in Software Development,” Tealfeed, 2022.